Ex.No.: Date:	UNIFIED MODELING LANGUAGE (UML)

The Unified Modeling Language (UML) is a general-purpose modeling language in the field of software engineering, which is designed to provide a standard way to visualize the design of a system.

The Unified Modeling Language (UML) offers a way to visualize a system's architectural blueprints in a diagram (see image), including elements such as

- Any activities (jobs)
- Individual components of the system and how they can interact with other software components.
- How the system will run
- How entities interact with others (components and interfaces)
- External user interface

Although originally intended solely for object-oriented design documentation, the Unified Modeling Language (UML) has been extended to cover a larger set of design documentation (as listed above) and has been found useful in many contexts.

The Unified Modeling Language (UML) is a general-purpose modeling language in the field of software engineering, which is designed to provide a standard way to visualize the design of a system.

The Unified Modeling Language (UML) offers a way to visualize a system's architectural blueprints in a diagram (see image), including elements such as

- Any activities (jobs)
- Individual components of the system and how they can interact with other software components.
- How the system will run
- How entities interact with others (components and interfaces)
- External user interface

Although originally intended solely for object-oriented design documentation, the Unified Modeling Language (UML) has been extended to cover a larger set of design documentation (as listed above) and been found useful in many contexts.

PROBLEM DEFINITION

Problem definition is a statement that describes the state of the problem that will come in the way of software development and also the ways to overcome the problem with a suitable solution.

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

A **software requirements specification (SRS)** is a description of a software system to be developed, laying out functional and non-functional requirements, and may include a set of use cases that describe interactions the users will have with the software.

An example organization of an SRS is as follows:

- Introduction
- Purpose
- Scope

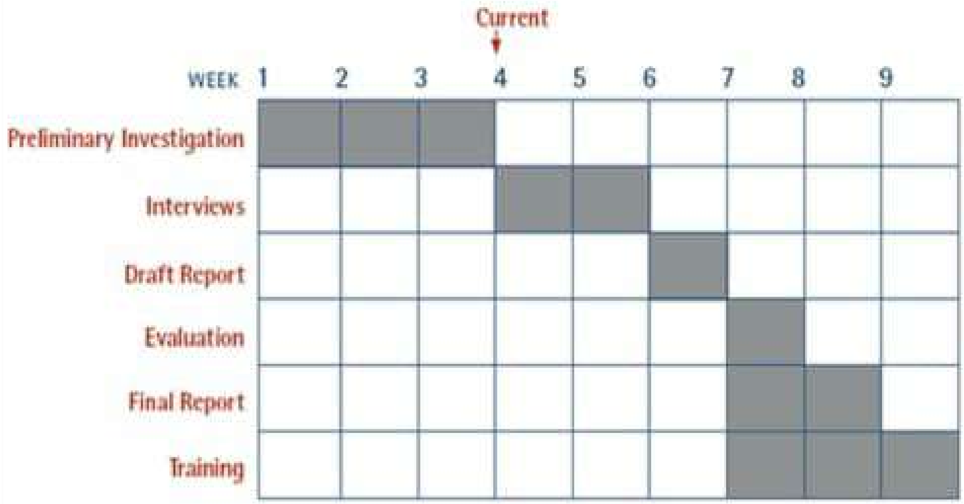
An example organization of an SRS is as follows:

- Introduction
- Purpose
- Scope
- Definition, Acronyms and
- Abbreviations
- Reference
- Technology to be used
- Tools to be used
- Overview
- Overall description
- Productive description
- Software interface
- Hardware interface
- System function
- User Characteristic
- Constraints
- Assumption and Dependences

GANTT CHART

A **Gantt chart** is a type of bar chart, first developed by Karol Adamiecki in 1896, and independently by Henry Gantt in the 1910s, that illustrates a project schedule. Gantt charts illustrate the start and finish dates of the terminal elements and summary elements of a project.

The Following Diagram Illustrates The Basic Gantt Chart



UML DIAGRAMS

The following UML diagrams describe the process involved in the software development

UML DIAGRAMS

The following UML diagrams describe the process involved in the software development process.

- Use case diagram
- Class diagram
- Sequence diagram
- Collaboration diagram
- State chart diagram
- Activity diagram
- Component diagram
- Deployment diagram
- Package diagram

USE CASE DIAGRAM

The use case is shown as an ellipse containing the name of the use case. An actor is shown as a stick figure with the name below it. Use case diagram is a graph of actors.

A use-case is a typical interaction between a user and a system that captures user goals & needs. Expressing these high-level processes and interactions it is referred to as use-case modeling. Once the use case model is better understood and developed we should start to identify classes and create their relationships.

CLASS DIAGRAM

The class diagram represents the types of objects in the system and the various kinds of static relationships that exist between them. Class diagrams are used in object modeling where real-world objects are mapped to logical objects in a computer program.

A class is drawn as a rectangle box with three compartments or components separated by horizontal lines. The top compartment holds the class name and middle compartment holds the attribute and the bottom compartment holds a list of operations.

Notations and symbols used in Class diagrams are

1) Class Notation
2) Object Diagram
3) Class Interface Notation
4) Binary Association Notation and Association Role
5) Qualifier
6) Multiplicity
7) OR Association
8) Association Class
9) N – ARY Association
10) Aggregation and Composition
11) Generalization.

SEQUENCE DIAGRAM

A sequence diagram shows an interaction arranged in a time sequence. It shows objects participating in interaction by their lifeline by the message they exchange arranged in time sequence. The vertical dimension represents time and the horizontal dimension represents an object.

11) Generalization.

SEQUENCE DIAGRAM

A sequence diagram shows an interaction arranged in a time sequence. It shows objects participating in interaction by their lifeline by the message they exchange arranged in time sequence. The vertical dimension represents time and the horizontal dimension represents an object.

COLLABORATION DIAGRAM

A collaboration diagram is similar to a sequence diagram but the message is in number format. In a collaboration diagram sequence diagram is indicated by the numbering of the message. A collaboration diagram, also called a communication diagram or interaction diagram, A sophisticated modeling tool can easily convert a collaboration diagram into a sequence diagram and the vice versa. A collaboration diagram resembles a flowchart that portrays the roles, functionality, and behavior of individual objects as well as the overall operation of the system in real-time.

STATE CHART DIAGRAM

The state chart diagram contains the states in the rectangle boxes and starts in indicated by the dot and the finish is indicated by the dot encircled. The purpose of a state chart diagram is to understand the algorithm in the performing method.

ACTIVITY DIAGRAM

An activity diagram is a variation or special case of a state machine in which the states or activity representing the performance of the operation and transitions are triggered by the completion of the operation. The purpose is to provide a view of the close and what is going on inside a use case or among several classes.

COMPONENT DIAGRAM

The component diagram is represented by figure dependency and it is a graph of the design of figure dependency. The component diagram's main purpose is to show the structural relationships between the components of a system. It is represented by a boxed figure. Dependencies are represented by communication associations.

DEPLOYMENT DIAGRAM

It is a graph of nodes connected by communication association. It is represented by a three-dimensional box. A deployment diagram in the unified modeling language serves to model the physical deployment of artifacts on deployment targets. Deployment diagrams show "the allocation of artifacts to nodes according to the Deployments defined between them. It is represented by a 3-dimensional box. Dependencies are represented by communication associations.

PACKAGE DIAGRAM

A package diagram is represented as a folder shown as a large rectangle with a top attached to its upper left corner. A package may contain both sub-ordinate package and ordinary model elements. All UML models and diagrams are organized into packages. A package diagram in unified modeling language that depicts the dependencies between the packages that make up a model. A Package Diagram (PD) shows a grouping of elements in the OO model and is a Cradle extension to UML. PDs can be used to show groups of classes in Class Diagrams (CDs), groups of components or processes in Component Diagrams (CPDs), or groups of processors in Deployment Diagrams (DPDs).

There are three types of layers. They are

1. User interface layer
2. Domain layer
3. Technical services layer